LAYER 5

FAILSAFE / VETO LAYER

Vertical Decomposition & Agentic Coding IDE Rulebook

DAO Governance Architecture

Self-Evolving & Self-Optimizing Quadratic Voting Module

Document Type:	Technical Specification & Implementation Rulebook
Target Audience:	Agentic Coding IDE / Smart Contract Developers
Classification:	Layer 5 — Failsafe / Veto Layer
Date:	May 2026

TABLE OF CONTENTS

1. ARCHITECTURAL NORTH STAR

Agent Context Anchor

PROMPT RULE: Before generating any code for Layer 5, the agent **MUST** restate: *"Layer 5 is purely subtractive. It cannot create, mint, transfer, or execute. It can only delay, veto, or revert."*

Principle	Agentic Constraint
Subtractive Only	Generated functions MUST NOT contain execute, call, delegatecall, or state-changing operations
Time-Gated	Every parameter adjustment MUST pass through a TimelockQueue with minimum delay.
Ejection Safety	Veto trigger MUST atomically cancel pending adjustments + restore baseline parameters.
Non-Recursive	Layer 5 MUST NOT call back into Layer 3 (Intelligence Layer). No feedback loops.

2. VERTICAL COMPONENT STACK

```
Layer 5
■■■ 5.1 Timelock Controller (The Delay)
■ ■■■ 5.1.1 Proposal Queue
■ ■■■ 5.1.2 Delay Counter
■ ■■■ 5.1.3 Execution Window
■■■ 5.2 Veto Authority (The Circuit Breaker)
■ ■■■ 5.2.1 Multisig Registry
■ ■■■ 5.2.2 Ragequit Gateway
■ ■■■ 5.2.3 Veto Vote Tally
■■■ 5.3 Baseline Registry (The Safe State)
■ ■■■ 5.3.1 Static Parameter Snapshot
■ ■■■ 5.3.2 Baseline Commit Hash
■ ■■■ 5.3.3 Restoration Engine
■■■ 5.4 Audit & Observability (The Black Box)
■■■ 5.4.1 Event Emitter
■■■ 5.4.2 State Diff Logger
■■■ 5.4.3 Veto Justification Store
```

3. GRANULAR COMPONENT SPECIFICATIONS

Agent Rules & Implementation Contracts

3.1 Timelock Controller (TimelockController.sol)

ID	Rule	Severity	Agent Test
T1	MIN_DELAY is immutable after deployment. If agent suggests making it mutable, REJECT.	■ Critical	ASSERT(MIN_DELAY == deployedValue)
T2	queueAdjustment() MUST emit AdjustmentQueued(id, params, executableAt) before returning.	■ Critical	Event emission test
T3	executeAdjustment() MUST revert if block.timestamp < executableAt.	■ Critical	Timestamp boundary test
T4	executeAdjustment() MUST check !isVetoed(id) before execution.	■ Critical	State dependency test
T5	Only Layer 3 (Intelligence Layer) address can call queueAdjustment(). Use onlyIntelligenceLayer modifier.	■ Critical	Access control test

AGENT TEMPLATE — Do not deviate from this interface

```
interface ITimelockController { event AdjustmentQueued(bytes32 indexed id, ParameterSet
params, uint256 executableAt); event AdjustmentExecuted(bytes32 indexed id); event
AdjustmentVetoed(bytes32 indexed id, address vetoer, string justification); function
queueAdjustment(ParameterSet calldata params) external onlyIntelligenceLayer returns (bytes32
id); function executeAdjustment(bytes32 id) external onlyAfterDelay(id) onlyIfNotVetoed(id);
function cancelAdjustment(bytes32 id) external onlyVetoAuthority; // AGENT RULE: No function
may bypass the delay. No emergency execution. }
```

3.2 Veto Authority (VetoAuthority.sol)

ID	Rule	Severity	Agent Test
V1	Veto power is held by a multisig with THRESHOLD >= 60% of steward [illegible] MIN_STEWARDS=7.	CRITICAL	Deployment validation
V2	veto() MUST accept justification string (min 20 chars). Empty justification MUST REVERT.	HIGH	Input validation test
V3	veto() MUST call BaselineRegistry.restore() immediately upon successful veto.	CRITICAL	Integration test
V4	ragequit() allows token holders to exit with pro-rata treasury share ONLY if VetoCrisis = true for 7 days.	HIGH	State condition test
V5	Veto authority CANNOT call queueAdjustment(). Separation of powers enforced at interface level.	CRITICAL	Interface segregation test

AGENT TEMPLATE

```
interface IVetoAuthority { struct VetoRecord { bytes32 adjustmentId; address[] signers; string justification; uint256 timestamp; } function veto(bytes32 adjustmentId, string calldata justification) external onlyMultisigThreshold; function ragequit() external onlyDuringVetoCrisis; // AGENT RULE: No steward may veto twice on same adjustmentId. mapping(bytes32 => mapping(address => bool)) public hasVetoed; }
```

3.3 Baseline Registry (BaselineRegistry.sol)

ID	Rule	Severity	Agent Test
B1	Baseline parameters are set at constructor() and stored as immutable. baselineHash =	Critical	Hash verification
B2	restoreToBaseline() MUST be callable by VetoAuthority AND TimelockController (if	Required	Access control test
B3	Restoration MUST overwrite ALL dynamic parameters in a single atomic transaction.	Critical	Atomicity test
B4	Post-restore, currentConfigHash MUST equal immutableBaselineHash	Critical	State equality test
B5	Restoration emits BaselineRestored(restorer, block.timestamp, restoredHash)	Required	Event test

AGENT TEMPLATE

```
interface IBaselineRegistry { event BaselineRestored( address indexed restorer, uint256 timestamp, bytes32 paramsHash ); function restoreToBaseline() external onlyAuthorizedRestorer; function getBaselineHash() external view returns (bytes32); function getCurrentHash() external view returns (bytes32); // AGENT RULE: baselineParams is a frozen snapshot. Never modify after deployment. ParameterSet public immutable baselineParams; }
```

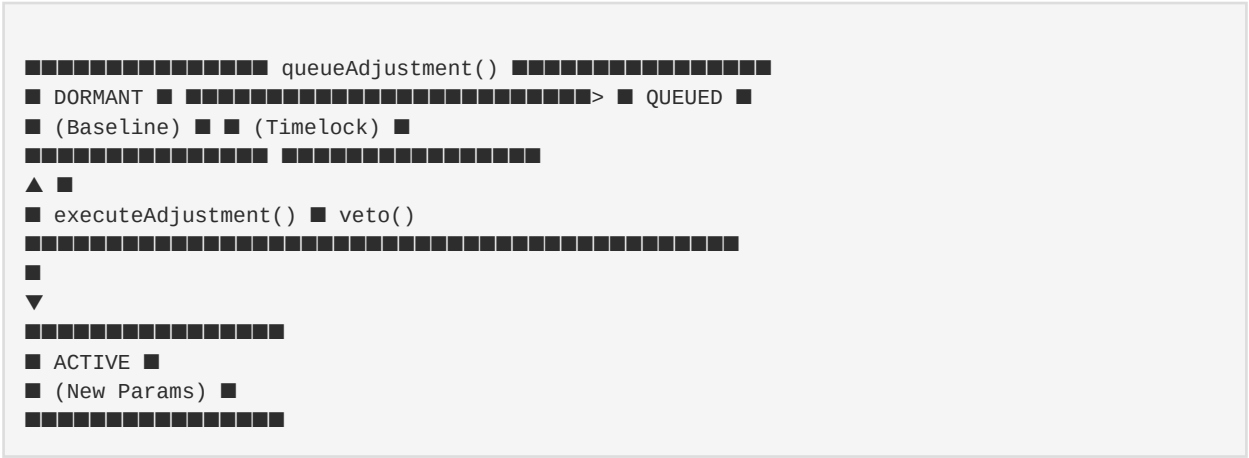
3.4 Audit & Observability (FailsafeLogger.sol)

ID	Rule	Severity	Agent Test
L1	Every state change in Layer 5 MUST emit an event. Silent state changes are FORBIDDEN.	■ Required	Event coverage test
L2	logStateDiff() MUST record beforeHash and afterHash for every parameter adjustment.	■ Required	Diff integrity test
L3	Veto justifications are stored on-chain (not just emitted) for permanent audit trail.	■ Required	Storage verification
L4	Logger MUST NOT have selfdestruct or upgrade capability. Append-only.	■ Critical	Code absence test

4. STATE MACHINE

Agent Implementation Guide

The system operates on an autonomous adaptation cycle. Layer 5 enforces the following state transitions:



AGENT RULE: Implement as enum Layer5State { DORMANT, QUEUED, ACTIVE, VETOED }. No additional states permitted.

5. CROSS-LAYER INTERACTION RULES

Critical for Agent Implementation

These rules govern how Layer 5 interacts with other layers in the DAO Governance Architecture. Violations of these rules compromise the entire safety model.

Interaction	Rule	Violation Consequence
Layer 3 → 5	Layer 3 calls queueAdjustment() ONLY. Never directly modifies parameters.	If Layer 3 bypasses timelock, Layer 5 is compromised.
Layer 5 → 2	Layer 5 calls restoreToBaseline() on Layer 2's parameter store.	Layer 5 NEVER calls Layer 2's execute functions.
Layer 5 → 5	Internal calls only between Timelock → Veto → Baseline Registry → Monitor.	Prevents external oracle manipulation.

6. AGENTIC IDE PROMPT INJECTION RULES

Context Block for Agent Prompting

When prompting your agent to build Layer 5, prepend these constraints to ensure safe code generation:

```
[LAYER_5_CONTEXT] You are building the Failsafe/Veto Layer of a DAO Governance system. INVARIANTS YOU MUST ENFORCE: 1. No function in this layer may initiate a proposal, transfer value, or execute external calls except restoreToBaseline(). 2. All parameter changes MUST sit in a timelock for MIN_DELAY seconds. 3. Veto authority is a multisig (>=7 members, >=60% threshold). 4. Veto triggers automatic, atomic restoration to baseline parameters. 5. This layer is append-only and non-upgradeable. 6. Every function MUST emit events. No silent operations. FORBIDDEN PATTERNS: - delegatecall to untrusted addresses - upgradeable proxy patterns - recursive calls to Layer 3 - execution bypass functions ("emergencyExecute", "fastTrack", etc.) - mutable MIN_DELAY [/LAYER_5_CONTEXT]
```

7. TESTING CHECKLIST FOR AGENT

Mandatory Test Cases

Generate tests that verify the following invariants. Each test MUST pass before the code is considered valid:

[1] test_CannotExecuteBeforeDelay() — Reverts if executed early

[2] test_VetoRestoresBaseline() — Veto triggers BaselineRestored event

[3] test_VetoCannotInitiate() — Veto authority cannot call queueAdjustment()

[4] test_RagequitOnlyDuringCrisis() — Ragequit reverts if <3 vetoes in 7 days

[5] test_BaselineHashImmutable() — Deployment hash matches restoration hash

[6] test_NoSilentStateChanges() — Every state change emits event

[7] test_JustificationRequired() — Empty string reverts veto

[8] test_Layer3CannotBypassTimelock() — Direct parameter changes from Layer 3 are rejected

8. FILE STRUCTURE

Agent Scaffold

Use the following directory structure when scaffolding Layer 5 contracts:

```
contracts/ ■■■ layer5/ ■ ■■■ TimelockController.sol # 5.1 ■ ■■■ VetoAuthority.sol # 5.2 ■ ■■■ BaselineRegistry.sol # 5.3 ■ ■■■ FailsafeLogger.sol # 5.4 ■ ■■■ interfaces/ ■ ■■■ ITimelockController.sol ■ ■■■ IVetoAuthority.sol ■ ■■■ IBaselineRegistry.sol ■■■ layer2/ # (existing, read-only for Layer 5) ■ ■■■ ParameterStore.sol ■■■ layer3/ # (existing, only queueAdjustment caller) ■■■ IntelligenceLayer.sol
```

9. QUICK REFERENCE: AGENT DECISION MATRIX

Accept / Reject Scenarios

Use this matrix to evaluate user requests during agentic code generation for Layer 5:

Scenario	Agent Action	Rationale
Add "emergency execution"	■ **REJECT**	Violates T1, T3 — No bypass allowed
Make MIN_DELAY mutable	■ **REJECT**	Violates T1 — Immutable safety boundary
Let veto authority propose	■ **REJECT**	Violates V5 — Separation of powers
Add Layer 3 callback after veto	■ **REJECT**	Violates Non-Recursive principle
Store veto justification off-chain	■ **REJECT**	Violates L3 — On-chain audit trail required
Add timelock for Layer 5 itself	■ **ACCEPT**	Self-governance is allowed
Add event for veto	■ **ACCEPT**	Already required by L1
Add ragequit cooldown period	■ **ACCEPT**	Enhances safety without violating rules
Add veto justification minimum length	■ **ACCEPT**	Enhances accountability
Allow partial baseline restore	■ **REJECT**	Violates B3 — Must be atomic
Add veto expiration timer	■ **ACCEPT**	Reasonable timeout mechanism
Allow single-steward veto	■ **REJECT**	Violates V1 — Multisig threshold required

Summary: Layer 5 is the ultimate safety net. Every design decision must prioritize *subtractive power, time-gated execution, and atomic restoration* over convenience or flexibility. When in doubt, reject the request and cite the governing rule.

APPENDIX: RULE ID QUICK REFERENCE

Category	Rule IDs	Contract
Timelock	T1, T2, T3, T4, T5	TimelockController.sol
Veto	V1, V2, V3, V4, V5	VetoAuthority.sol
Baseline	B1, B2, B3, B4, B5	BaselineRegistry.sol
Logging	L1, L2, L3, L4	FailsafeLogger.sol

This document was generated as a comprehensive rulebook for agentic coding environments. All rules, interfaces, and constraints are derived from the DAO Governance Architecture's Layer 5 specification.